ESCOLA TÈCNICA SUPERIOR
D'ENGINYERIA
Universitat Rovira i Virgili

Memòria de Pràctiques Externes

Autor: **David Diestre Rubio**
Organització: **SEES LAB**
Tutor Acadèmic: **Carles Barberà Escoí**

Data: **11/2026**

Contents

1 Introduction

1.1 Toward High-Level Molecular Design

The long-term vision of molecular artificial intelligence is to enable systems that can propose chemically valid molecules from high-level specifications. In practice, this would look like a "property-first" interface: a researcher describes the desired behaviour of a compound (e.g., solubility, toxicity constraints, binding affinity, thermal stability), and the model outputs one or more candidate molecules that satisfy those constraints.

If this vision becomes practical, it could significantly speed up research in multiple domains such as drug discovery and materials design.

A key step for achieving this feat is proper representation of molecules. Many models operate directly on symbolic strings such as SMILES, but SMILES is primarily a syntax for encoding graphs; it is not a direct representation of physicochemical structure or property-relevant features. Therefore, the need arises for a useful intermediate representation that is compact, machine-friendly, and more closely aligned with structural information.

Molecular fingerprints are one widely used intermediate abstraction. They compress structural information into a fixed-length vector, and they are commonly used in similarity search and property prediction. Reconstructing molecules from fingerprints is therefore a fundamental building block toward property-conditioned molecular generation: it is the step that translates a property-aligned vector space back into a concrete, synthesizable molecule.

However, an important limitation of this framework is that the relationship between molecular fingerprints and molecular structures is not bijective. Multiple distinct molecules can share the same fingerprint representation, and conversely, small variations in structure may not be fully captured by the fingerprint encoding. This intrinsic ambiguity makes the reconstruction problem ill-posed and highlights the need for machine learning models capable of learning probabilistic mappings from the fingerprint space to valid molecular structures. Such models can capture the underlying distribution of molecules compatible with a given representation, enabling robust and chemically meaningful generation.

1.2 MolForge and Its Reported Performance

MolForge is a transformer-based model designed to reconstruct SMILES strings from 2048-bit molecular fingerprints [1]. The official repository reports strong reconstruction metrics, including high validity rates, high Tanimoto similarity [1] between original and reconstructed fingerprints, and competitive exact reconstruction performance.

These results are promising, but a single benchmark table is not enough to understand how reliable the model really is. In particular, if a model is to be used as a component in a larger "design loop", we need to know more than its reported performance; we must validate its results against the same data that was used by its authors, as well as against different, probably less favorable, cases. This leads to the following questions:

- Can the reported numbers be reproduced by an independent setup?
- Does performance transfer to molecules outside the evaluation dataset?
- Do the chosen metrics reflect what we actually care about (structural identity), or can they be misleading?
- How fragile is the model to small perturbations of its input representation?

This report studies these questions through a controlled assessment of MolForge as a black-box inference system, focusing on molecules outside its training distribution and on controlled fingerprint perturbations.

1.3 Research Questions

The report is structured around four research questions:

1. **Reproducibility (RQ1).** Can MolForge's reported performance be reproduced under controlled, independent experimental conditions?
2. **Generalization across molecular regimes (RQ2).** How does performance change as evaluation moves from existing molecules to increasingly novel molecular regimes? In particular, how does the model perform on novel generated but chemically valid molecules?
3. **Robustness (RQ3).** How sensitive is MolForge to controlled bit-level perturbations in the 2048-bit fingerprint representation?
4. **Metric adequacy (RQ4).** Is Tanimoto similarity sufficient to measure reconstruction quality? How does it compare to stricter metrics such as canonical SMILES exact match?

[1] Tanimoto similarity is a standard way of measuring how similar two molecular fingerprints are. It is computed by comparing the bits that are active in both fingerprints relative to the total number of active bits across them. Its value ranges from 0 to 1, where 1 means that the two fingerprint representations are identical.

2 Background

2.1 Molecular Representations

Molecules can be represented in several ways, each with different trade-offs in terms of structural fidelity and suitability for machine learning. SMILES is a compact linear serialization of the molecular graph, which makes it convenient for sequence-based models and standard cheminformatics pipelines. Figure 1 shows one example of how a molecular structure can be expressed as a character sequence. However, although SMILES encodes molecular connectivity, it remains primarily a symbolic syntax rather than a representation directly aligned with physicochemical structure or property-relevant features. In this work, molecular datasets are standardized using RDKit canonical SMILES, a deterministic form that assigns a unique string to a molecule, since the same molecular graph can be written as multiple valid SMILES.

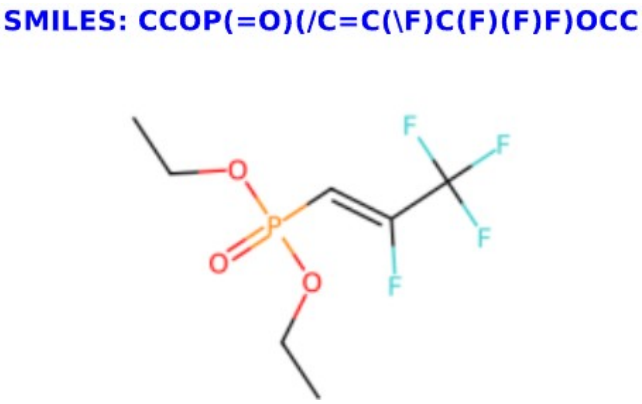


Figure 1: Example of a SMILES string and its corresponding 2D molecular depiction.

Graph representations are more explicit in terms of molecular structure, but they still do not directly provide a compact representation of the property-relevant features we care about for prediction or generation tasks. For many molecular machine learning tasks, fingerprints provide a more practical intermediate representation. In this work, we consider several RDKit fingerprint families, including MACCS, Avalon, path-based fingerprints, atom-pair and topological torsion fingerprints, and circular fingerprints such as ECFP and FCFP. Our main focus is on 2048-bit hashed ECFP4 fingerprints, since MolForge reports its strongest reconstruction performance with this representation. ECFP4 fingerprints encode local circular substructures into a fixed-length bit vector, which makes them especially suitable as model inputs. At the same time, we compare similarity metrics across the broader set of fingerprint types to assess how evaluation depends on the chosen molecular representation. However, regardless of the specific fingerprint family, this convenience comes at a cost: the representation is *not invertible*. Different molecules can share the same or similar fingerprint because of hashing collisions and because fingerprints intentionally discard part of the molecular information, including aspects of global structural context. Consequently, reconstructing a unique molecule from a fingerprint is fundamentally ill-posed.

2.2 Evaluation Metrics

Evaluating fingerprint-to-SMILES reconstruction requires more than one metric, since each of them captures a different aspect of performance. In this work, we use four complementary metrics:

- **Validity:** fraction of predicted SMILES that parse into a chemically valid molecule.
- **Canonical SMILES exact match:** fraction of predictions that match the original structure after canonicalization.
- **Tanimoto similarity:** similarity between fingerprints of the original and predicted molecules.
- **Tanimoto exact match:** fraction of predictions whose fingerprint has Tanimoto similarity exactly equal to 1 with the original one.

Each metric answers a different question. Validity measures whether the decoder produces chemically well-formed outputs, but it does not say whether the structure is correct. Exact match is strict and interpretable, but it can be "too strict" if the task allows multiple valid reconstructions for the same fingerprint. Tanimoto similarity is widely used because it provides a graded notion of similarity, but it can be misleading in high-dimensional binary spaces: a molecule can differ in chemically meaningful ways while only changing a relatively small fraction of bits. Tanimoto exactness is useful for assessing how easily two fingerprints can coincide perfectly. The purpose of studying it is to show that it should not be blindly interpreted as a strong success metric on its own, since its values typically come out higher than canonical SMILES exact match: different molecules may still produce identical fingerprint representations.

2.3 Overview of MolForge and CoCoGraph

MolForge. A transformer-based sequence model that maps a fingerprint representation to a SMILES sequence [1]. In this project it is treated as a black box: we do not retrain it, change its architecture, or tune hyperparameters. The goal is not to improve the model, but to stress-test its reported behaviour under independent evaluation conditions.

CoCoGraph. A graph-based generative model that produces chemically valid molecules [2]. We use CoCoGraph datasets as a controlled way to increase distribution shift, moving from already existing molecules to generated ones that explore synthetic but valid chemical space. This allows us to assess how MolForge performs under a realistic novel-molecule setting.

3 Experimental Framework

3.1 Reproducible Computational Setup

A major goal of this work was to make the experiments easy to reproduce for someone cloning the repository and following the same workflow. For that reason, the setup has been intentionally designed to be simple and portable:

- **CPU-only execution** ensures that the evaluation does not depend on specific GPU availability, although a provisional AI-generated notebook based on the CPU one was also used to allow faster GPU execution.
- **Separated Conda environments** avoid dependency conflicts (MolForge inference vs. RDKit-heavy preprocessing/analysis).
- **Two execution contexts** were supported (local WSL and remote lab environment) to be able to work both on a personal machine and on the provided lab computers.

All experiments were executed in batches of 2000 molecules per condition. This batch size provides a practical compromise: it is large enough to produce stable aggregate metrics, while still being small enough to run multiple conditions (datasets and noise levels) within reasonable compute time (40-60 min per run).

3.2 Used Datasets

Generalization cannot be summarized by a single number, since it depends on *how far* the evaluation distribution is from the model's training distribution. To make this explicit, we define three dataset regimes representing increasing distribution shift.

3.2.1 MolForge training distribution

This dataset contains molecules drawn from MolForge's provided evaluation distribution. It serves as an upper-bound baseline, and it is the correct place to ask the reproducibility question (RQ1).

3.2.2 External real molecules (outside MolForge provided data)

To test real-world generalization, we evaluate molecules not explicitly belonging to MolForge's released splits. We consider two sources: (i) PubChem samples [3], and (ii) real molecules used to train CoCoGraph [2]. Both are composed of existing molecules, but they are outside the test data provided in MolForge's repository. Because PubChem is orders of magnitude larger than typical training sets, the probability of overlap is low; therefore, a performance drop here would suggest that MolForge's released evaluation data is reasonably unbiased, while also indicating the presence of a genuine distribution shift and/or reliance on memorization patterns.

3.2.3 Novel molecules generated by CoCoGraph

Finally, we evaluate on chemically valid molecules generated by CoCoGraph [2]. These molecules do not correspond to real existing compounds, therefore they provide a useful test setting for assessing MolForge under a realistic molecular generation scenario. In particular, they allow us to study whether MolForge could function reliably as a general decoder within future pipelines aimed at generating novel molecules with desired properties.

3.3 Experimental Protocol (end-to-end pipeline)

For each dataset regime, we run two input conditions. First, we evaluate *clean* fingerprints to measure baseline reconstruction. Second, we evaluate *noisy* fingerprints generated through controlled bit flips under three different settings: flipping only zero bits, flipping only one bits, and applying random flips. This allows us to assess whether errors affecting one bits, which are typically scarcer and more informative, have a stronger impact on reconstruction. This noise study is not only a stress test: it is also intended to emulate the kind of perturbations that may naturally arise in a molecular generation pipeline based on machine learning, where intermediate representations are not expected to be perfectly preserved.

For each run, we compute validity, canonical exact match, Tanimoto similarity, and Tanimoto exact match. In addition, the code generates qualitative PDF reports that allow a domain expert to visually inspect failures: the original molecule, the reconstructed molecule, and where structural divergence occurs, in order to assess how visually significant small changes in Tanimoto coefficient or SMILES exactness are in a real molecular structure.

Algorithmic summary (end-to-end evaluation)

1. Standardize input SMILES.
2. Convert SMILES to 2048-bit ECFP4 fingerprints (RDKit).
3. Optionally inject controlled noise (bit flips at rate p).
4. Run MolForge inference: fingerprint (ECFP4) $\rightarrow$ predicted SMILES.
5. Compute metrics (validity, SMILES exact match, Tanimoto, Tanimoto exactness).
6. Export quantitative tables/plots and qualitative PDF comparisons.

4 Implementation and Repository Architecture

4.1 Repository Structure (why it is organized this way)

A practical difficulty in projects of this kind is that preprocessing, model execution, and analysis can easily end up mixed together in a way that is difficult to maintain or reproduce. The repository was therefore organized to keep these stages clearly separated and to make the workflow easier to rerun and extend. The main idea is to keep the project ordered, with each step having a clear role within the overall pipeline.

At a high level:

- `envs/` contains separate environment definitions for MolForge execution and analysis.
- `data/` stores raw SMILES sources, model inputs (fingerprints), model outputs (predicted SMILES), and analysis artifacts (tables/plots/PDFs).
- `notebooks/` provides a readable narrative for each stage (preprocessing, inference, analysis), without mixing them.
- `src/` contains reusable utilities (SMILES conversion, fingerprints).

Repository tree

```
MolForge_Testing/
├─ envs/
│  ├─ molforge/environment.yml  (MolForge env; CPU inference)
│  └─ tools/environment.yml     (RDKit + analysis env)
├─ data/
│  ├─ SMILES/raw/       (MolForge, CoCoGraph, PubChem)
│  ├─ MolForge_input/   (2048-bit fingerprints; clean/noisy)
│  ├─ MolForge_output/  (predicted SMILES)
│  └─ analysis_output/  (tables, plots, PDFs)
├─ notebooks/
│  ├─ preprocessing/
│  │  ├─ noise.ipynb                      (add noise)
│  │  ├─ raw_to_SMILES.ipynb              (raw → SMILES)
│  │  └─ SMILES_to_MolForge_input.ipynb   (SMILES → FP)
│  ├─ MolForge/
│  │  ├─ data/sp/                      (tokenizer vocab)
│  │  ├─ saved_models/                 (checkpoints)
│  │  └─ fps_to_smiles_MolForge.ipynb  (FP → SMILES)
│  └─ analysis/
│     └─ output_analysis.ipynb  (metrics + plots)
├─ src/
│  ├─ smiles_to_fp.py  (SMILES → FP utils)
│  └─ fingerprints.py  (RDKit mol → SMILES)
└─ .gitignore
```

4.2 Fingerprint Generation and Noise Injection (preprocessing)

Fingerprints are computed using RDKit's Morgan fingerprint algorithm [4]. In particular, we use ECFP4 with a 2048-bit hashed representation, as this configuration corresponds to the setting for which MolForge reports its best performance.

Noise injection is implemented as controlled random bit flips. This design is simple but informative: it lets us precisely control the severity of perturbations and plot a response curve (noise rate vs. metric degradation).

Noise model (bit flips)

Given a fingerprint $f \in \{0,1\}^{2048}$, we sample $m_i \sim \mathrm{Bernoulli}(p)$ independently for $i = 1, \ldots, 2048$ and flip bits where $m_i = 1$.

4.3 MolForge Integration

Although MolForge is treated as a black-box model [1], correct evaluation still requires matching the input formatting expected by the repository. The integration therefore focuses on:

- Loading the official pretrained checkpoints without modification.
- Replicating the expected fingerprint input format.
- Keeping track of invalid inputs or outputs, as well as possible runtime errors.

4.4 Analysis

Beyond reporting a single average, the analysis notebook generates several complementary outputs intended to make the evaluation interpretable. In particular, it produces per-molecule comparison files, aggregated result tables, plots, and qualitative PDF pages.

A key point is that inference is always performed from **ECFP4** fingerprints, since this is the fingerprint setting for which MolForge reports its best results. However, evaluation is not limited to ECFP4 alone. To obtain a broader view of reconstruction quality, the Tanimoto coefficient is computed for all fingerprint families considered in the study. This is done by taking both the original molecule and the reconstructed molecule, passing them through RDKit, and encoding them in each fingerprint space before computing similarity.

At the quantitative level, the notebook generates detailed per-instance outputs together with aggregated tables by dataset and noise level. These tables report metrics such as average Tanimoto similarity, exact Tanimoto matches, canonical exact matches, and the percentage of invalid outputs. Aggregates are computed both by discarding invalid generations and by counting them as zero in the Tanimoto score, allowing us to distinguish between invalid outputs and incorrect reconstructions.

The notebook also generates plots summarizing the effect of perturbations on performance. In particular, the noise experiments are turned into response curves showing how the main metrics evolve as the bit-flip rate increases under different corruption settings. These figures are useful because they make the degradation trends much easier to interpret than a collection of raw tables alone.

Finally, a qualitative PDF report is produced to support visual inspection. Each selected page compares the original molecule and the reconstructed molecule side by side, together with the corresponding textual and similarity information. This is important because some reconstruction errors are much easier to understand visually than from a scalar metric alone.

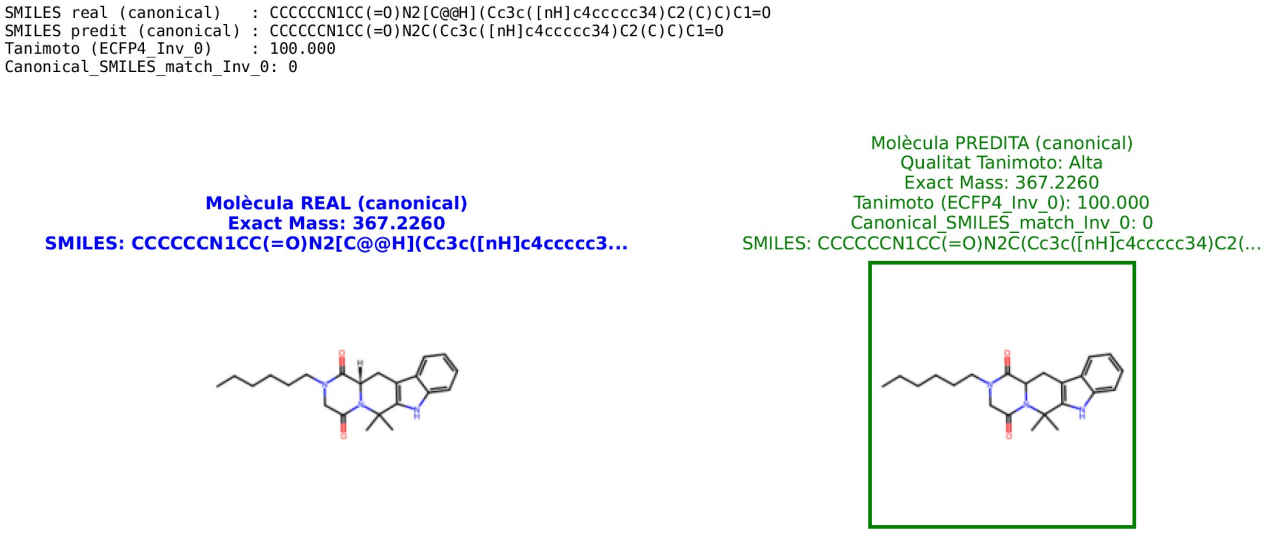


Figure 2: Example page from the qualitative PDF report generated during analysis.

4.5 GitHub Repository

The complete codebase used for preprocessing, inference, and analysis is available in the project repository:

<https://github.com/DavidDiestreR/MolForge_Testing.git>

5 Experimental Investigation

This section answers the four research questions introduced in Section 1. The goal is not only to report a final scoreboard, but to understand under which conditions MolForge works well, where it starts to fail, and which metrics are actually informative for judging reconstruction quality.

Throughout this section, four aggregate metrics are reported: average Tanimoto similarity across the fingerprint families considered in this work (`Avg_Tc`), exact fingerprint agreement (`Tc-exacts`), canonical SMILES exact match, and invalid generations. In addition, the Tanimoto values for each individual fingerprint representation can be inspected in `data/analysis_output/[dataset-specific_folder]/results`.

5.1 RQ1: Reproducibility

The first question is whether MolForge's reported strong performance can be reproduced under controlled conditions. This is first checked on MolForge's own official test split, but also on two additional datasets of real molecules: an independent PubChem sample and molecules coming from the CoCoGraph training pool.

This is important because if only the official MolForge split worked well, one could argue that the reported baseline depends too much on the specific evaluation data distributed with the repository. By contrast, if similar numbers appear on independent but comparable molecules, then the original reported performance is much more credible.

Table 1: Clean reconstruction results on non-novel datasets when invalid predictions are counted as zero in the average Tanimoto score.

Dataset	Avg_Tc (%)	Tc-exacts (%)	Canonical exact (%)	Invalid (%)
MolForge official test split	96.35	86.55	64.05	0.20
PubChem sample	95.25	83.65	64.35	0.60
CoCoGraph training molecules	96.65	87.00	68.85	0.55

Table 2: Clean reconstruction results on non-novel datasets when invalid predictions are excluded from the average Tanimoto score.

Dataset	Avg_Tc (%)	Tc-exacts (%)	Canonical exact (%)	Invalid (%)
MolForge official test split	96.54	86.55	64.18	0.20
PubChem sample	95.83	83.65	64.74	0.60
CoCoGraph training molecules	97.18	87.00	69.23	0.55

The official MolForge test split reproduces the expected strong baseline: average similarity is above 96%, exact fingerprint agreement is around 86.5%, and invalid generations are almost negligible. More importantly, the two additional datasets show very similar behaviour, with average similarity above 95% and very low invalid rates. This confirms that the strong performance is not limited to the official split, but also holds on independent yet similar molecular data.

It is also worth noting that canonical exact match, which is not used as an evaluation metric in the official MolForge repository results, is clearly lower than the Tanimoto-based exactness metric in our evaluation. This stricter criterion gives a noticeably lower score, which already suggests that the two metrics are not capturing the same notion of reconstruction success.

In this case, differences between the two averaging conventions are negligible, as invalid generations are rare in these datasets and have minimal impact on the results.

Overall, RQ1 can be answered positively. The strong results reported for MolForge are reproducible, and they remain consistent on independent but comparable datasets.

5.2 RQ2: Generalization across molecular regimes

The second question asks what happens once evaluation moves outside MolForge's familiar regime. For this, the key case is the set of novel molecules generated by CoCoGraph, which represent a much stronger distribution shift than the previous datasets.

Table 3: Clean reconstruction results on novel generated molecules.

Treatment of invalid outputs	Avg_Tc (%)	Tc-exacts (%)	Canonical exact (%)	Invalid (%)
Invalid predictions counted as zero in Avg_Tc	79.95	44.60	36.00	4.15
Invalid predictions excluded from Avg_Tc	83.41	44.60	37.56	4.15

Here the drop is clear. Compared with the three datasets used in RQ1, average similarity decreases substantially, exact fingerprint matches are almost cut in half, canonical exact reconstruction falls to roughly one third of the molecules, and invalid outputs increase by almost one order of magnitude.

This provides the strongest indication in the study that MolForge's performance depends heavily on how close the evaluation molecules are to the molecular regimes represented during training. It performs strongly on real molecules, but becomes much less reliable on genuinely novel generated ones.

The difference between the two averaging conventions also becomes more relevant here. In the previous datasets, excluding invalid generations barely changed the average similarity. For novel molecules, the change is larger because invalid outputs are no longer rare.

RQ2 is therefore answered clearly: MolForge generalizes much worse on the novel molecules generated by CoCoGraph, and the results suggest that the model is more reliable on already existing molecules than on genuinely novel ones.

5.3 RQ3: Robustness to fingerprint perturbations

The third question studies robustness to controlled bit-level perturbations in the 2048-bit fingerprint.

The full noise analysis was run on two representative datasets only: MolForge's official test split and the novel molecules generated by CoCoGraph. This choice was deliberate. The clean-data results in RQ1 already showed that MolForge's official test split, the PubChem sample, and the CoCoGraph training molecules behave very similarly. Because of that, MolForge's own test data can be used as a fair in-distribution representative without repeating the full noise sweep on all three. The CoCoGraph novel molecules then provide the complementary hard case under strong distribution shift.

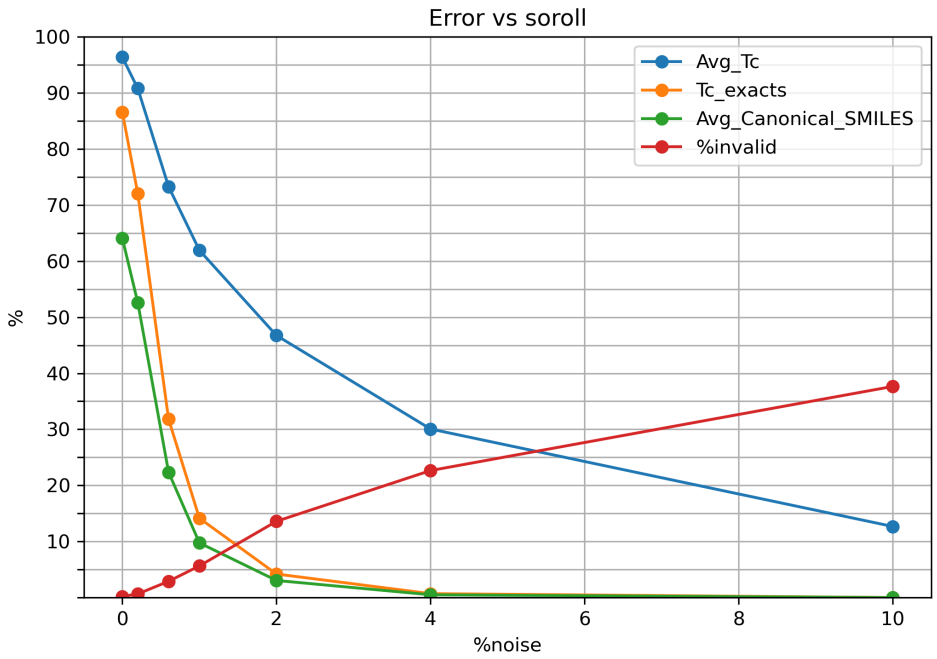
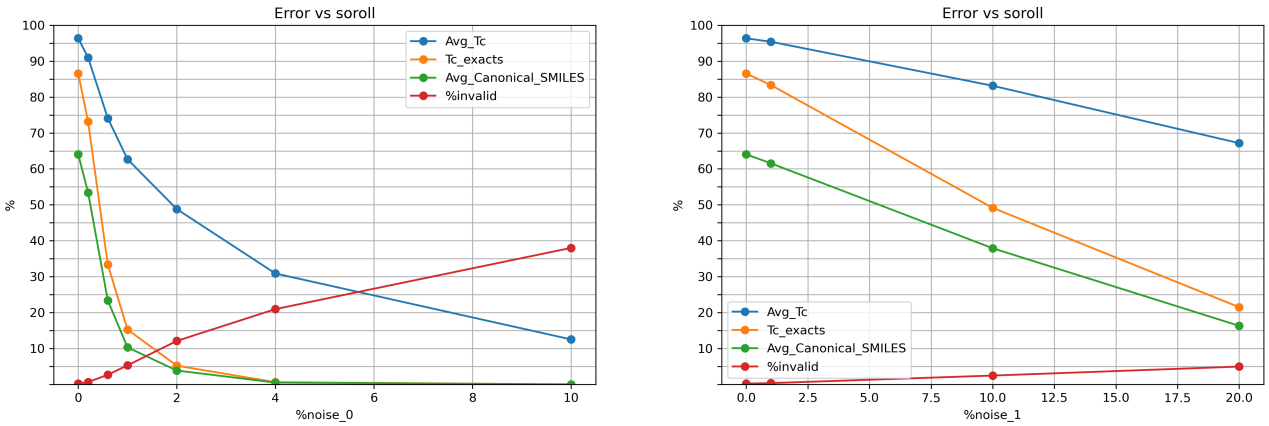


Figure 3: Performance degradation under increasing random fingerprint noise on MolForge's official test split.

Figure 3 shows that performance according to average Tanimoto similarity degrades consistently as noise increases, but not all metrics degrade in the same way. Average similarity falls progressively, while exact fingerprint agreement and canonical exact match collapse much earlier. Invalid generations also rise steadily.

In practice, this means that partial similarity can still remain high even after the exact molecular identity has already been lost. So robustness looks much weaker when judged by strict reconstruction criteria than when judged by average similarity alone.



(a) Noise applied on zero bits ($0 \to 1$).

(b) Noise applied on one bits ($1 \to 0$).

Figure 4: Asymmetric robustness analysis on MolForge's official test split.

The asymmetric analysis can be especially informative but must be interpreted carefully. Activating bits that were originally zero produces a much faster degradation curve than deleting bits that were originally one. However, this does not necessarily mean that changing a one is intrinsically less harmful than changing a zero.

A key point is that the perturbations are applied as percentages within each group. Since sparse fingerprints such as ECFP4 contain many more zero bits than one bits, the same percentage of $0 \to 1$ noise corresponds to a much larger number of bit flips than the same percentage of $1 \to 0$ noise. Therefore, the observed asymmetry reflects not only the type of perturbation, but also the strong imbalance between inactive and active bits. What the results clearly show is that, under this percentage-based noise model, flipping zeros to ones leads to a much faster degradation of reconstruction quality.

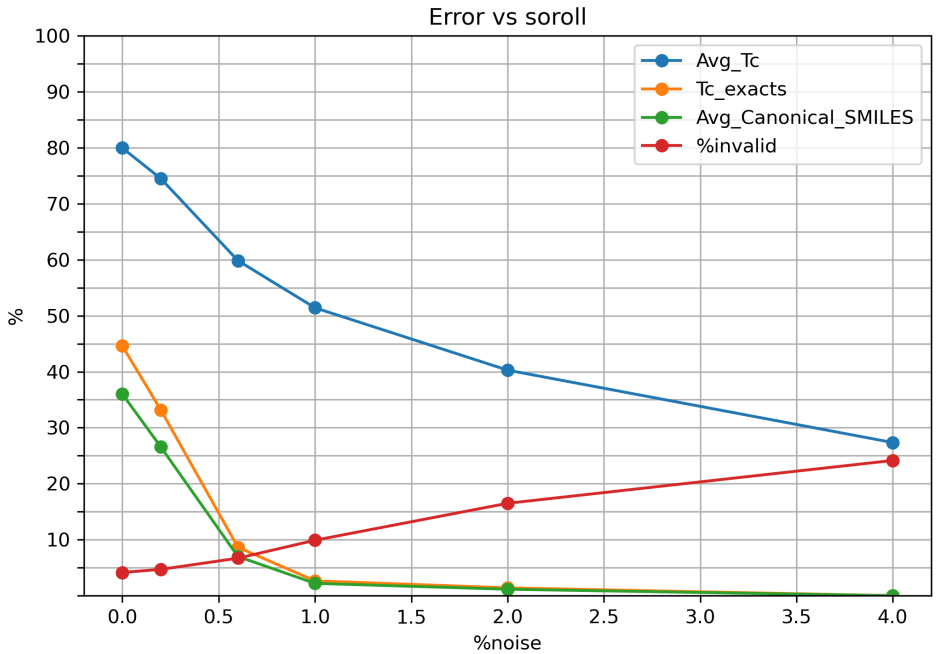


Figure 5: Performance degradation under increasing random fingerprint noise on CoCoGraph novel molecules.

On the novel generated molecules, the same qualitative pattern appears, but from a much weaker starting point. Since clean reconstruction is already harder in this regime, even small perturbations push the exact metrics close to zero and increase invalid generations much faster.

This means that noise and distribution shift reinforce one another. Molecules that are already difficult to decode under clean conditions become extremely fragile once the fingerprint is no longer exact.

RQ3 shows that, as expected for any learned decoder, MolForge is sensitive to noise in the input fingerprint representation. In practice, however, the degradation in exact reconstruction is remarkably steep: with only 0.66% noise, exactness-based metrics already drop to around one third of their clean-data value, and even further in the case of novel molecules, while average Tanimoto similarity declines much more gradually. This once again shows that similarity-based metrics can make the loss of reconstruction quality appear milder than it actually is in terms of exact molecular recovery.

5.4 RQ4: Metric adequacy

The final question is whether Tanimoto similarity alone is sufficient to evaluate reconstruction quality.

The answer is no. Across the experiments above, Tanimoto similarity is useful to measure broad structural closeness, but it does not reliably indicate whether the original molecule has been exactly recovered.

This is already visible in the clean-data setting, where average similarity remains very high while exact reconstruction metrics are clearly lower. The same issue becomes even more evident in the noise experiments: Tanimoto decreases progressively, but exact fingerprint agreement and canonical exact match collapse much earlier. Thus, similarity-based evaluation can make the degradation look softer than it really is in terms of exact recovery.

This limitation is especially important in the novel-molecule setting. There, average similarity may still suggest partially acceptable reconstructions, while the exact metrics and invalid rates reveal a much harsher failure mode. Looking only at Tanimoto would therefore overestimate model quality precisely in the most challenging scenarios.

RQ4 is therefore answered clearly: Tanimoto similarity is informative, but not sufficient on its own. It should be complemented with exact-match metrics and explicit reporting of invalid generations in order to evaluate fingerprint-to-SMILES reconstruction properly.

6 Conclusion

6.1 Results

The experimental results draw a coherent picture of MolForge's behaviour across the different evaluation settings.

First, MolForge proves to be a strong decoder on non-novel molecules, and this behaviour is reproducible beyond the official split distributed with the repository. Similar performance is obtained on independent datasets of real existing molecules, which supports the credibility of the reported baseline.

Second, performance degrades substantially on the novel molecules generated by CoCo-Graph. This indicates that generalization becomes much weaker under stronger distribution shift, and suggests that the model is considerably more reliable on already existing molecules than on genuinely novel ones.

Third, the robustness analysis shows that MolForge is sensitive to perturbations in the fingerprint representation. Even small amounts of noise lead to a sharp drop in exact reconstruction, while average Tanimoto similarity decreases more gradually.

Finally, the comparison between metrics shows that Tanimoto similarity alone is not sufficient to assess reconstruction quality. Similarity-based scores often remain relatively high even when exact fingerprint agreement, canonical exact match, and validity already reveal a much stronger loss of reconstruction quality.

Overall, MolForge works well in favourable regimes, but becomes much less reliable under novelty and noisy inputs. This is especially relevant if the model is intended to be used as the final decoding stage of a molecular generation pipeline, where fingerprints may come from approximate upstream predictions rather than from exact known molecules. In that setting, novelty and small representation errors are likely to appear naturally, so the present results suggest that MolForge can still be a useful decoder, but with clear room for improvement in robustness and in its ability to handle genuinely novel molecular structures.

6.2 Learning Acquired from the Internship

This internship gave me practical experience not only with the technical tools commonly used in computational research, such as GitHub, Jupyter notebooks, Conda environments, and the organization of a complete Python project, but also with the less obvious side of research work: making a project reproducible, dealing with setup issues, adapting workflows to different execution environments, and learning how to turn an initially messy exploration into a structured experimental pipeline.

Beyond the technical side, this was the first project in which I had to report progress regularly, discuss intermediate results with a supervisor, and decide how to proceed when the next step was not always obvious. Because of that, I learned to work in a more disciplined and iterative way: not only running experiments, but also stopping to assess whether the results were actually meaningful, whether the methodology was fair, and whether a different direction was worth exploring.

On a more personal level, the internship taught me that progress in this kind of work usually comes from iteration rather than from one decisive step. Much of the process involved identifying problems, refining the setup, reconsidering earlier choices, and gradually turning the workflow into something more solid and interpretable. Since the work had to be sustained week after week, it also forced me to organize my time better and to work with more continuity instead of relying on isolated bursts of progress. This made me more critical with my own results and more consistent in the way I approached a longer-term technical project.

6.3 Personal Assessment and Observations of Interest

I found the internship to be a very valuable experience. One of my main reasons for doing it was to understand more clearly how research in machine learning works in practice, beyond the simplified image one often has from papers or coursework, and in that sense the experience fully met my expectations.

What I found most interesting is that research is not only about proposing new ideas, but also about carefully examining existing ones. A significant part of the work involves understanding previous methods, reproducing their reported behaviour, identifying where they remain strong and where they start to fail, and being critical about what the chosen evaluation actually shows. This gave me a much more realistic view of research as a process of questioning, contrasting, and refining, rather than simply producing results.

Overall, I finish the internship with a very positive impression. In addition to the technical knowledge I acquired, I gained a better understanding of how research projects evolve over time, how important it is to interpret results cautiously, and how much of the work depends on revisiting decisions, refining the methodology, and discussing results critically.

References

[1] KNU-LCBC, *Molforge: Fingerprint-to-smiles reconstruction model (repository)*, GitHub repository, 2024. [Online]. Available: https://github.com/knu-lcbc/MolForge.

[2] M. Ruiz, *Cocograph: Graph-based molecular generation (repository)*, GitHub repository, 2024. [Online]. Available: https://github.com/manurubo/CoCoGraph.

[3] National Center for Biotechnology Information, *Pubchem*, Online database, 2025. [Online]. Available: https://pubchem.ncbi.nlm.nih.gov.

[4] G. Landrum, *Rdkit: Open-source cheminformatics*, Documentation, 2025. [Online]. Available: https://www.rdkit.org.